



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Basi di dati

Lezioni, e Simulazioni d'esame

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

settembre 2026

Indice

1	Le Basi di Dati e i loro Utenti	3
1.1	Cos'è una Base di Dati	3
1.2	DBMS e Database System	3
1.3	Architettura di un Database System	3
1.4	Cosa Deve Fornire un DBMS	4
1.5	Interazione tra Applicazioni e Basi di Dati	4
1.6	Le Origini del Modello Relazionale	4
1.7	Indipendenza dei Dati	4
1.7.1	Indipendenza Logica dei Dati	5
1.7.2	Indipendenza Fisica dei Dati	5
1.8	Livelli di Astrazione dei Dati	5
1.9	Perché Usare una Base di Dati	6
1.9.1	Condivisione dei Dati e Transazioni Multi-utente	6
1.10	Digressione: OLTP vs. OLAP	6
1.11	Utenti di una Base di Dati	7
1.11.1	Tipi di Utilizzatori	7
1.12	Quando è Meglio NON Usare un DBMS	7
2	Il Modello Relazionale	8
2.1	Concetti Generali	8
2.2	Cos'è una Relazione	8
2.2.1	Definizione Intensionale	8
2.2.2	Definizione Estensionale	9
2.3	Stato di una Relazione	9
2.4	Chiave di una Relazione	9
2.5	Terminologia	9
2.6	Vincoli	9
2.7	Vincolo di Chiave	10
2.8	Vincolo di Integrità dell'Entità (Entity Integrity)	10
2.9	Vincoli di Integrità Referenziale	11
2.10	Schema di un Database Relazionale	12
2.11	Altri Tipi di Vincolo	12
2.12	Stato di una Base di Dati Relazionale	12
2.13	Operazioni di Aggiornamento e Violazione dei Vincoli	12
2.13.1	Violazioni per Operazione INSERT	13
2.13.2	Violazioni per Operazione DELETE	13
2.13.3	Violazioni per Operazione UPDATE	13
2.14	Come Preservare l'Integrità Referenziale	13
3	Algebra Relazionale	14
3.1	Introduzione	14
3.2	Schema e Dati di Riferimento	15
3.3	Selezione (SELECT, σ)	15
3.4	Proiezione (PROJECT, π)	16
3.5	Unione, Differenza e Intersezione	16
3.6	Prodotto Cartesiano ($\times$)	16
3.7	Ridenominazione (RENAME, ρ)	16

3.8	Join	16
3.8.1	Theta-Join	16
3.8.2	Join Naturale	17
3.8.3	Equijoin	17
3.9	Divisione ($\div$)	17
3.10	Outer Join	17
3.11	Espressioni ed Alberi di Query	18

1 Le Basi di Dati e i loro Utenti

1.1 Cos'è una Base di Dati

Una **base di dati** (BdD) è una raccolta organizzata (strutturata) di informazioni che devono essere gestite per un periodo di tempo lungo, anche di molti anni. Più precisamente, una base di dati è una raccolta di informazioni gestita da un **database management system** (DBMS).

Esempi di Basi di Dati:

- Dati relativi a studenti, corsi ed esami di un'università
- Il catalogo dei prodotti di un marketplace elettronico
- I *Points of Interest* (POI) su una mappa (es. Google Maps)
- Il Pubblico Registro Automobilistico (PRA) dell'Automobile Club Italiano (ACI)
- I pagamenti delle tasse dei contribuenti di un Comune
- Gli orari dei voli e le prenotazioni dei passeggeri di una compagnia aerea

1.2 DBMS e Database System

- **Database Management System (DBMS)**: pacchetto software progettato per gestire basi di dati informatizzate.
- **Database System**: l'insieme costituito dal software del DBMS insieme ai dati memorizzati nella base di dati. Talvolta nella definizione sono incluse anche le applicazioni software esterne che interagiscono con il DBMS per eseguire i propri compiti.

1.3 Architettura di un Database System

- Gli **utenti non esperti** (*unsophisticated*: clienti, agenti di viaggio, ...) interagiscono tramite *Web Forms* o *Application Front Ends*; gli **utenti esperti** (*sophisticated*: programmatori, amministratori) interagiscono tramite un'*SQL Interface*. Entrambi generano comandi SQL.
- Il **Query Evaluation Engine** riceve i comandi ed è composto da *Parser*, *Optimizer*, *Plan Executor* e *Operator Evaluator*.
- Il livello sottostante gestisce *Files and Access Methods*, il *Buffer Manager* e il *Recovery Manager*, coordinati dal blocco di **Concurrency Control** (*Transaction Manager* e *Lock Manager*).
- Alla base, il *Disk Space Manager* gestisce fisicamente gli **Index Files**, i **Data Files** e il **System Catalog** che compongono il database.

1.4 Cosa Deve Fornire un DBMS

Un DBMS deve:

- supportare la memorizzazione di grandi quantità di dati;
- gestire la robustezza dei dati e il loro recupero in caso di fallimenti, errori o azioni malevole;
- controllare l'accesso ai dati di più utenti contemporaneamente (**concorrenza**), evitando interazioni indesiderate tra utenti (*isolation*) e modifiche incomplete dei dati (*atomicity*).

Per la gestione dei dati sono previsti diversi linguaggi specializzati:

- **DDL** (*Data Definition Language*): definisce gli schemi per strutturare i dati;
- **DML** (*Data Manipulation Language*, o *query language*): modifica i dati e li estrae dalla base di dati;
- **DCL** (*Data Control Language*): gestisce l'accesso ai dati (**grant**, **revoke**, ...);
- **TCL** (*Transaction Control Language*): gestisce la sicurezza delle transazioni (**commit**, **rollback**, ...).

1.5 Interazione tra Applicazioni e Basi di Dati

I programmi applicativi interagiscono con la base di dati generando:

- **Interrogazioni** (*queries*): accesso a differenti porzioni di dati e formulazione della risposta;
- **Transazioni** (*transactions*): sequenze atomiche di azioni (letture/scritture) sulla base di dati.

Le applicazioni non devono consentire l'accesso ai dati ad utenti non autorizzati.

1.6 Le Origini del Modello Relazionale

Un enorme impulso allo sviluppo delle basi di dati è venuto dal lavoro di **Ted Codd** (1970), che ha introdotto l'idea di mostrare agli utenti una **visione logica dei dati**, indipendente dalla struttura fisica. Le interrogazioni vengono così espresse attraverso un linguaggio di alto livello, che utilizza il modello logico e non quello fisico dei dati.

1.7 Indipendenza dei Dati

La separazione tra programmi e dati è spesso chiamata **indipendenza programmi-dati**: permette di modificare le strutture dati e l'organizzazione fisica dei dati senza dover cambiare l'accesso dei programmi alla base di dati via DBMS.

Nei sistemi basati su file, invece, la struttura dei file di dati è contenuta nei programmi che devono accedervi: se cambia la struttura del file, è probabile che debbano essere modificati anche i programmi che vi accedono, rendendo difficile e costosa la manutenzione del software nel tempo.

1.7.1 Indipendenza Logica dei Dati

- **Definizione:** capacità di modificare lo schema logico del database senza dover alterare lo schema esterno o le applicazioni che lo utilizzano.
- **Vantaggi:** consente di aggiungere, rimuovere o modificare campi e tabelle senza influire sulle applicazioni esistenti; migliora la flessibilità e la gestione dell'evoluzione del database.
- **Esempio:** aggiungere un nuovo attributo a una tabella senza modificare il codice delle applicazioni che vi accedono.

1.7.2 Indipendenza Fisica dei Dati

- **Definizione:** capacità di modificare lo schema fisico del database senza influenzare lo schema logico o le applicazioni.
- **Vantaggi:** permette di ottimizzare le prestazioni (es. cambiando la struttura di memorizzazione o gli indici) senza alterare la logica delle applicazioni; migliora la scalabilità e la gestione delle risorse hardware.
- **Esempio:** spostare un database su un nuovo hardware, o ristrutturarne l'archiviazione, senza cambiare la modalità di accesso ai dati per l'utente finale.

1.8 Livelli di Astrazione dei Dati

- **Schema logico:** modello che rappresenta i dati nei termini del modello proprio del DBMS (es. il modello relazionale nei RDBMS);
- **Schema fisico:** dettagli su come le relazioni descritte dallo schema logico sono memorizzate su memoria secondaria (es. file di record ordinati/non ordinati, tipo di indici creati, ecc.);
- **Schema esterno:** una o più **viste** specifiche per velocizzare/facilitare/autorizzare l'accesso a insiemi di dati a specifici utenti o gruppi di utenti.

I tre livelli seguono il flusso: *livello fisico* $\rightarrow$ *livello logico* $\rightarrow$ *livello esterno (viste)*.

Esempio. Schema logico per un DB universitario:

```
Students(sid: string, name: string, login: string, age: integer, gpa: real)
Faculty(fid: string, fname: string, sal: real)
Courses(cid: string, cname: string, credits: integer)
Rooms(rno: integer, address: string, capacity: integer)
Enrolled(sid: string, cid: string, grade: string)
Teaches(fid: string, cid: string)
Meets_In(cid: string, rno: integer, time: string)
```

- **Schema fisico:** file non ordinato di record, con un indice B+Tree sulla prima colonna di STUDENTS e COURSES.
- **Schema esterno:** una relazione che mostri tutti i corsi con relativo docente e numero di iscritti: `Courseinfo(cid: string, fname: string, enrollment: integer)`.

1.9 Perché Usare una Base di Dati

- Indipendenza logica e fisica dei dati (vista sopra);
- Accesso concorrente ai dati;
- Crash recovery;
- Integrità dei dati e sicurezza;
- Amministrazione dei dati;
- Riduzione dei tempi di sviluppo di applicazioni.

1.9.1 Condivisione dei Dati e Transazioni Multi-utente

La condivisione dei dati permette a gruppi di utenti di leggere e scrivere contemporaneamente (*concurrent users*) sulla stessa base di dati:

- il **controllo della concorrenza** nel DBMS garantisce che ogni transazione sia eseguita correttamente o annullata;
- il **sottosistema di recovery** assicura che ogni transazione correttamente completata sia memorizzata in modo permanente.

Questo permette di eseguire centinaia di transazioni concorrenti al secondo mantenendo validità e coerenza dei dati: si parla di **OLTP** (*Online Transaction Processing*).

1.10 Digressione: OLTP vs. OLAP

	OLTP	OLAP
Finalità	Gestire le transazioni quotidiane di un'azienda (es. ordini, pagamenti)	Supportare l'analisi complessa dei dati per BI e reporting
Operazioni	Alta frequenza, operazioni brevi e semplici (insert , update , delete)	Bassa frequenza, query complesse e di lunga durata
Priorità	Integrità/coerenza dei dati; risposta rapida alle query	Query ad-hoc, analisi multidimensionale
Schema	Normalizzato, per minimizzare la ridondanza	Denormalizzato (es. <i>star schema</i>), per le performance
Esempi	E-commerce, banche, punti vendita	Reporting aziendale, dashboard di BI

Tabella 1: Confronto tra sistemi OLTP e OLAP

Sistemi per OLTP:

- **Database Relazionali (RDBMS)**: MySQL, MariaDB, PostgreSQL, Microsoft SQL Server, IBM DB2, Oracle Database;

- **Software ERP:** SAP, Oracle, Panthera, TeamSystem, ...

Sistemi per OLAP:

- **Data Warehousing Platforms:** Amazon Redshift, Google BigQuery, Snowflake, Microsoft Azure Synapse Analytics;
- **Business Intelligence Tools:** Tableau, Power BI, Qlik Sense, SAP BusinessObjects.

1.11 Utenti di una Base di Dati

Gli utenti si dividono in due gruppi principali:

- chi utilizza e controlla il contenuto della base di dati, insieme a chi progetta e sviluppa applicazioni su di essa (**utilizzatori**);
- chi progetta e sviluppa il software del DBMS con tutti i suoi sottosistemi (**sviluppatori**).

1.11.1 Tipi di Utilizzatori

- **Database Administrator** (Amministratore): responsabile dell'autorizzazione di accesso alla base di dati, del coordinamento e monitoraggio del suo utilizzo, dell'acquisto di risorse software/hardware, del monitoraggio dell'efficienza operativa del sistema.
- **Database Designer** (Progettista): responsabile della definizione del contenuto, dello schema, dei vincoli e delle transazioni eseguibili sulla BD; comunica con gli utenti per raccoglierne le necessità.
- **End-users** (Utenti finali): utilizzano i dati mediante query e report; alcuni possono aggiornare i contenuti. Si suddividono in:
 - *Casual* (Occasionali): accedono alla base di dati solo occasionalmente, quando necessario (es. top manager che leggono dati fortemente aggregati);
 - *Naïve o Parametric* (non-esperti): la maggioranza degli utenti finali; usano funzioni predefinite (le *canned transactions*) per interagire con la BD (es. cassieri di banca, impiegati di segreteria, utenti di social media o applicazioni web).

1.12 Quando è Meglio NON Usare un DBMS

Cosa può inibirne l'adozione:

- alti investimenti iniziali e necessità di hardware dedicato;
- costi aggiuntivi per sicurezza, controllo di concorrenza, recovery, funzioni di integrità.

Quando un DBMS potrebbe non essere necessario:

- se la base di dati e l'applicazione sono molto semplici, ben definite e non soggette a modifiche continue;
- se non è richiesto l'accesso di più utenti.

Quando adottarlo può essere impossibile o sconsigliato:

- in molti *embedded systems*, dove mancano adeguate capacità di memorizzazione per un DBMS *general purpose*;
- con requisiti di tempo reale molto stringenti (es. sistemi di switch telefonico);
- quando il modello dati del DBMS non è abbastanza espressivo per la complessità dei dati (es. basi di dati genomiche/proteiche);
- se servono operazioni speciali non supportate dallo specifico DBMS (es. GIS e sistemi basati sulla localizzazione).

2 Il Modello Relazionale

2.1 Concetti Generali

Il modello relazionale fu proposto da **E.F. Codd** (IBM Research) nel 1970, nel famoso articolo "*A Relational Model for Large Shared Data Banks*" (Communications of the ACM, giugno 1970). Il modello si basa sul concetto matematico di **relazione**, fondato sulla teoria degli insiemi. Questo lavoro fu alla base di una profonda rivoluzione nel campo delle basi di dati, che portò Codd a vincere il prestigioso **ACM Turing Award**.

2.2 Cos'è una Relazione

Informalmente, una **relazione** può essere vista come una tabella con un insieme di valori su ogni riga. Due livelli la definiscono:

- lo **schema** della relazione (livello *intensionale*);
- le **istanze** della relazione (livello *estensionale*).

2.2.1 Definizione Intensionale

Lo schema di una relazione definisce:

- il nome della relazione (es. STUDENTI);
- il nome di ogni attributo (es. SID, NOME, LOGIN, ...);
- il dominio di ogni attributo (es. INTEGER, STRING, ...), ovvero l'insieme dei valori che quell'attributo può assumere.

L'ordine degli attributi nello schema non ha alcun significato specifico. Notazione standard:

`Students(sid: string, name: string, login: string, age: integer, gpa: real)`

Il numero di attributi definisce il **grado** (o *arietà*) della relazione (5 nell'esempio sopra).

2.2.2 Definizione Estensionale

Un'**istanza** di uno schema di relazione è un insieme di tuple (o record), ognuna con lo stesso numero di campi dello schema:

- essendo un insieme, **non possono esserci duplicati**;
- l'ordine delle tuple non ha alcun valore specifico.

Il numero di istanze (tuple) definisce la **cardinalità** della relazione.

2.3 Stato di una Relazione

Lo stato di una relazione è un sottoinsieme del prodotto cartesiano dei domini dei suoi attributi. Data una relazione $R(A_1, A_2, \dots, A_n)$, uno specifico stato di R è definito come:

$$r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$$

ovvero $r(R) = \{t_1, t_2, \dots, t_n\}$ dove ogni t_i è una tupla, $t_i = \langle v_1, v_2, \dots, v_n \rangle$ e ogni v_j è un elemento di $\text{dom}(A_j)$. Nello stato di una relazione l'ordine delle tuple non conta (è un insieme!).

Esempio. Sia $R(A_1, A_2)$ con $\text{dom}(A_1) = \{0, 1\}$ e $\text{dom}(A_2) = \{a, b, c\}$. Il prodotto $\text{dom}(A_1) \times \text{dom}(A_2)$ è l'insieme di tutte le coppie ordinate possibili:

$$\{\langle 0, a \rangle, \langle 0, b \rangle, \langle 0, c \rangle, \langle 1, a \rangle, \langle 1, b \rangle, \langle 1, c \rangle\}$$

Un possibile stato $r(R)$ è $\{\langle 0, a \rangle, \langle 0, b \rangle, \langle 1, c \rangle\}$, con cardinalità 3.

2.4 Chiave di una Relazione

Ogni riga di una relazione ha un campo (o un insieme di campi) il cui valore identifica univocamente quella riga: si parla di **chiave** (*key*) della relazione. Talvolta si usano valori convenzionali per identificare una riga: si parla in questo caso di **chiavi artificiali** (*artificial key*) o **chiavi surrogate** (*surrogate key*).

2.5 Terminologia

Informale	Formale
Tabella	Relazione
Intestazione colonna	Attributo
Tutti i possibili valori di una colonna	Dominio
Riga della tabella	Tupla
Definizione della tabella	Schema della relazione
Tabella popolata	Stato della Relazione

Tabella 2: Corrispondenza tra terminologia informale e formale

2.6 Vincoli

I vincoli determinano quali stati di una relazione sono ammissibili. Sono di tre tipi:

1. **Vincoli impliciti:** dipendono dal data model stesso (es. il modello relazionale non ammette liste come valore di un attributo);
2. **Vincoli basati sullo schema** (o espliciti): definiti nello schema con gli strumenti forniti dal modello (es. un vincolo di integrità referenziale);
3. **Vincoli applicativi o semantici:** vanno oltre il potere espressivo del modello e devono essere imposti a livello di programma applicativo (es. un libro deve essere restituito entro 30 giorni dal prestito).

I principali vincoli espliciti nel modello relazionale sono:

- vincolo di dominio;
- vincolo di chiave;
- vincolo di integrità delle entità;
- vincolo di integrità referenziale.

2.7 Vincolo di Chiave

- **Superchiave** di R : insieme di attributi SK tale che non esistono due tuple distinte t_1, t_2 di $r(R)$ con $t_1[SK] = t_2[SK]$; la condizione deve valere in ogni stato valido di R .
- **Superchiave minimale** (o **chiave candidata**): una superchiave tale che rimuovendo un qualsiasi attributo si ottiene un insieme che non è più superchiave.
- Una chiave è sempre una superchiave, ma non viceversa.

Esempio. La relazione STUDENT ha due chiavi candidate: $\{sid\}$ e $\{login\}$. L'insieme $\{sid, name\}$ è una superchiave ma *non* una chiave (contiene una chiave, ma non è minimale). Ogni insieme di attributi che contiene una chiave è ovviamente una superchiave.

Se una relazione ha più chiavi candidate, una viene scelta (tipicamente dal DBA) come **chiave primaria**: in genere la più piccola per numero di attributi; quando ciò non è possibile, possono intervenire fattori soggettivi. I valori della chiave primaria identificano univocamente ogni tupla e possono essere usati per riferirsi ad essa da un'altra relazione.

2.8 Vincolo di Integrità dell'Entità (Entity Integrity)

Nessuno degli attributi che compongono la chiave primaria PK di una relazione R può avere valore NULL in alcuna tupla di $r(R)$:

$$t[PK] \neq null \quad \forall t \in r(R)$$

Se PK ha più attributi, NULL non è ammesso per nessuno di essi. È comunque possibile imporre che anche altri attributi (non chiave primaria) non possano assumere valore NULL.

2.9 Vincoli di Integrità Referenziale

A differenza degli altri, un vincolo di integrità referenziale coinvolge **due relazioni**: una relazione **referenziante** R_1 e una **referenziata** R_2 . In R_1 un insieme di attributi FK (**chiave esterna**, *Foreign Key*) fa riferimento alla chiave primaria PK di R_2 : una tupla $t_1 \in R_1$ riferenzia $t_2 \in R_2$ se $t_1[FK] = t_2[PK]$.

Esempio. `Enrolled` è la relazione referenziante, `Students` quella referenziata: `Enrolled.sid` è la chiave esterna che riferenzia `Students.sid`.

I valori della FK possono essere:

1. uno dei valori del corrispondente attributo PK in R_2 , oppure
2. il valore NULL (ma solo se la FK non fa parte della chiave primaria di R_1).

Osservazioni:

- la FK può fare riferimento alla *stessa* relazione a cui appartiene la PK;
- gli attributi della FK non devono necessariamente avere lo stesso nome dei corrispondenti attributi della PK.

Esempio: database COMPANY. Vincoli di integrità referenziale tipici in uno schema aziendale:

- `EMPLOYEE.Dno` $\rightarrow$ `DEPARTMENT.Dnumber`
- `EMPLOYEE.Super_ssn` $\rightarrow$ `EMPLOYEE.Ssn` (la FK riferenzia la stessa relazione della PK — vedi Osservazione 1)
- `DEPARTMENT.Mgr_ssn` $\rightarrow$ `EMPLOYEE.Ssn`
- `DEPT_LOCATIONS.Dnumber` $\rightarrow$ `DEPARTMENT.Dnumber`
- `PROJECT.Dnum` $\rightarrow$ `DEPARTMENT.Dnumber`
- `WORKS_ON.Essn` $\rightarrow$ `EMPLOYEE.Ssn`; `WORKS_ON.Pno` $\rightarrow$ `PROJECT.Pnumber`
- `DEPENDENT.Essn` $\rightarrow$ `EMPLOYEE.Ssn`

Esercizio 1

Dati gli schemi $R(A, B)$, $S(B, C)$ con Primary Key (A) e Foreign Key (B) references S(B), cosa si può concludere?

- a) A non può essere NULL ma B sì
- b) A e B possono essere NULL
- c) La primary key è parte della dichiarazione di R , la foreign key è nella dichiarazione di S
- d) B è una chiave primaria o una chiave candidata per S

Soluzione: d) — per definizione, una chiave esterna deve fare riferimento a una chiave primaria (o comunque candidata) della relazione referenziata, altrimenti il vincolo di integrità referenziale non sarebbe ben definito.

2.10 Schema di un Database Relazionale

I vincoli di integrità referenziale coinvolgono più di una relazione: non basta quindi considerare lo schema di una singola relazione. Uno **schema di database relazionale** è un insieme S di schemi di relazione appartenenti alla stessa base di dati:

$$S = \langle \{R_1, R_2, \dots, R_n\}, IC \rangle$$

dove $R_1, \dots, R_n$ sono i nomi degli schemi di relazione e IC è l'insieme dei vincoli di integrità.

2.11 Altri Tipi di Vincolo

I **vincoli di integrità semantica** si riferiscono al significato dell'applicazione e non possono essere espressi nel modello in sé (es. "il numero massimo di ore che un dipendente può lavorare su tutti i progetti è di 46 ore per settimana"). Serve quindi un linguaggio per la specifica di vincoli: SQL-99 introduce *trigger* e *asserzioni*, non implementati in modo uniforme da tutti i DBMS. In MySQL si può usare **CHECK**:

```
CREATE TABLE Person (...,  
    Age int CHECK (Age >= 18));
```

2.12 Stato di una Base di Dati Relazionale

Uno stato di una base di dati relazionale con schema S è un insieme di stati di relazione $\{r_1, r_2, \dots, r_m\}$ tale che ogni r_i è uno stato di R_i e soddisfa i vincoli di integrità in IC . Uno stato di una base di dati è talvolta chiamato **istantanea** (*snapshot*) o **istanza** (*instance*). Uno stato che non rispetta i vincoli in IC è uno **stato non valido**.

2.13 Operazioni di Aggiornamento e Violazione dei Vincoli

Le principali operazioni per cambiare lo stato di una base di dati sono:

- **INSERT**: inserimento di una nuova tupla;
- **DELETE**: cancellazione di una tupla;
- **MODIFY**: modifica di un attributo di una tupla.

Queste operazioni non devono mai portare alla violazione di un vincolo di integrità: garantire questa condizione può richiedere di propagare automaticamente gli aggiornamenti. Se si verifica una violazione, le opzioni possibili sono:

- **RESTRICT** (*NO ACTION*, *REJECT*): impedire l'operazione;
- **CASCADE**, **SET NULL**, **SET DEFAULT**: avviare altri aggiornamenti per eliminare la violazione;
- eseguire routine specifiche definite dall'utente.

2.13.1 Violazioni per Operazione INSERT

L'INSERT può violare **tutti** i vincoli:

- **dominio**: il valore di un attributo non appartiene al dominio specificato;
- **chiave**: inserimento di una tupla il cui valore di chiave esiste già;
- **integrità referenziale**: valore di FK che non esiste come valore di PK nella relazione referenziata;
- **integrità dell'entità**: valore della chiave primaria della nuova tupla è NULL.

Esercizio 2

Un'operazione di INSERT in una relazione può produrre la violazione di quale vincolo? (Dominio / Chiave / Integrità referenziale / Integrità delle entità / Nessuna / Tutte)

Soluzione: Tutte — come visto sopra, l'INSERT è l'unica operazione che può potenzialmente violare ciascuno dei quattro vincoli contemporaneamente.

2.13.2 Violazioni per Operazione DELETE

Il DELETE può violare **solo** vincoli di integrità referenziale, se si rimuove una tupla la cui chiave primaria è referenziata da altre tuple.

Esercizio 3

Dato lo schema Enrolled(sid, cid, grade) e Students(sid, name, login, age, gpa), con sid chiave esterna di Enrolled verso Students(sid): la cancellazione di una qualsiasi tupla nella tabella ENROLLED non può produrre la violazione di alcun vincolo. Vero o Falso?

Soluzione: Vero — Enrolled è la relazione "figlia" (referenziante): eliminare un suo record non viola mai l'integrità referenziale. La violazione scatterebbe solo cancellando un record "padre" in Students ancora referenziato da altre tuple.

2.13.3 Violazioni per Operazione UPDATE

L'UPDATE può violare tutti i vincoli, a seconda dell'attributo modificato:

- update della **chiave primaria**: integrità referenziale;
- update di una **chiave esterna**: integrità referenziale;
- update di un attributo ordinario: vincoli di dominio, o vincoli UNIQUE/NOT NULL.

2.14 Come Preservare l'Integrità Referenziale

- **RESTRICT** (*NO ACTION*): rifiuta l'operazione;
- **CASCADE**: cancella tutte le tuple che referenziavano la PK della tupla cancellata/modificata;

- **SET NULL**: assegna NULL alla FK delle tuple che referenziavano la tupla cancellata/modificata;
- **SET DEFAULT**: assegna un valore di default alla FK delle tuple che referenziavano la tupla cancellata/modificata.

Attenzione: le opzioni SET NULL e SET DEFAULT *non* sono disponibili se la chiave esterna include attributi che sono anche chiave primaria della tabella referenziante (altrimenti si violerebbe l'integrità dell'entità).

Esempio in SQL.

```
CREATE TABLE Enrolled (
    studid  CHAR(20),
    cid     CHAR(20),
    grade   CHAR(10),
    PRIMARY KEY (studid, cid),
    FOREIGN KEY (studid) REFERENCES Students
        ON DELETE CASCADE
        ON UPDATE NO ACTION
);
```

3 Algebra Relazionale

3.1 Introduzione

L'**algebra relazionale** è un linguaggio di interrogazione formale e **procedurale**: ogni operatore prende in input una o due relazioni e produce in output una relazione (proprietà di **chiusura**). Questo permette di comporre e annidare espressioni, applicando un operatore al risultato di un altro. L'algebra relazionale è alla base dei linguaggi pratici come SQL.

Operatori di base (un insieme completo, da cui derivano tutti gli altri):

- Selezione (σ), Proiezione (π);
- Unione ($\cup$), Differenza ($-$);
- Prodotto Cartesiano ($\times$);
- Ridenominazione (ρ).

Operatori derivati (esprimibili in funzione dei precedenti, ma usati per comodità):

- Intersezione ($\cap$);
- Join ($\bowtie$): theta-join, equijoin, join naturale;
- Divisione ($\div$).

3.2 Schema e Dati di Riferimento

Negli esempi che seguono si userà questo schema:

Sailors(sid: integer, sname: string, rating: integer, age: real)
 Boats(bid: integer, bname: string, color: string)
 Reserves(sid: integer, bid: integer, day: date)

con le seguenti istanze di esempio:

S1 (istanza di Sailors)				S2 (istanza di Sailors)			
sid	sname	rating	age	sid	sname	rating	age
22	dustin	7	45.0	28	yuppy	9	35.0
31	lubber	8	55.5	31	lubber	8	55.5
58	rusty	10	35.0	44	guppy	5	35.0
				58	rusty	10	35.0

R1 (istanza di Reserves)		
sid	bid	day
22	101	10/10/96
58	103	11/12/96

Tabella 3: Istanze di esempio usate negli operatori seguenti

3.3 Selezione (SELECT, σ)

L'operatore di **selezione** σ permette di selezionare un sottoinsieme delle tuple di una relazione, applicando a ciascuna di esse una formula booleana F .

Espressione: $\sigma_F(R)$	
Schema	$R(X) \rightarrow X$
Istanza	$r \rightarrow \sigma_F(r) = \{t \mid t \in r \text{ AND } F(t) = \text{vero}\}$

Tabella 4: Definizione formale di SELECT

F si compone di predicati connessi da AND ($\wedge$), OR ($\vee$) e NOT ($\neg$). Ogni predicato è del tipo $A \theta c$ oppure $A \theta B$, dove:

- A e B sono attributi in X ;
- $c \in \text{dom}(A)$ è una costante;
- θ è un operatore di confronto, $\theta \in \{=, \neq, <, >, \leq, \geq\}$.

Esempio. Trovare in S2 tutti i sailor che hanno giudizio superiore a 8: $\sigma_{\text{rating} > 8}(S2)$.

Risultato: $\{\langle 28, \text{yuppy}, 9, 35.0 \rangle, \langle 58, \text{rusty}, 10, 35.0 \rangle\}$ — stesso numero di attributi, selezionate le due (sole) tuple che soddisfano $\text{rating} > 8$.

3.4 Proiezione (PROJECT, π)

L'operatore di **proiezione** π estrae, da ogni tupla della relazione di input, solo gli attributi specificati, eliminando le colonne non richieste. Essendo il risultato un insieme, eventuali **tuple duplicate vengono eliminate**: per questo il numero di tuple del risultato può essere inferiore a quello della relazione di partenza.

Esempio. Trovare nome e giudizio di tutti i sailor (istanza S2): $\pi_{sname, rating}(S2)$.

Risultato: $\{\langle yuppy, 9 \rangle, \langle lubber, 8 \rangle, \langle guppy, 5 \rangle, \langle rusty, 10 \rangle\}$.

3.5 Unione, Differenza e Intersezione

Questi operatori insiemistici richiedono che le due relazioni operande siano **union-compatibili**: stesso numero di attributi, con domini corrispondenti compatibili.

- $R \cup S$: tutte le tuple che appartengono a R o a S (o entrambe);
- $R - S$: le tuple che appartengono a R ma non a S ;
- $R \cap S$ (*derivato*): le tuple che appartengono sia a R sia a S ; $R \cap S = R - (R - S)$.

$\cup$ e $\cap$ sono **commutativi**; $-$ **non** lo è.

3.6 Prodotto Cartesiano ($\times$)

$R \times S$ combina **ogni** tupla di R con **ogni** tupla di S : lo schema del risultato è la concatenazione degli attributi di R e di S (con $|R| \cdot |S|$ tuple in output). Se R e S hanno un attributo con lo stesso nome, questo compare due volte nello schema risultato, creando ambiguità.

Esempio. $S1 \times R1$: 6 tuple (3×2), con la colonna **sid** duplicata (una da S1, una da R1) — l'ambiguità che motiva l'operatore di ridenominazione.

3.7 Ridenominazione (RENAME, ρ)

L'operatore ρ rinomina la relazione e/o i suoi attributi, senza modificare i dati, per risolvere ambiguità come quella del prodotto cartesiano.

Esempio. $\rho(C(1 \rightarrow sid1, 5 \rightarrow sid2), S1 \times R1)$: rinomina il 1° attributo (sid da S1) in **sid1** e il 5° (sid da R1) in **sid2**.

3.8 Join

3.8.1 Theta-Join

Il **theta-join** $R \bowtie_c S$ equivale a $\sigma_c(R \times S)$: un prodotto cartesiano seguito da una selezione secondo una condizione c qualsiasi (non solo uguaglianze). Lo schema del risultato mantiene **entrambi** gli attributi coinvolti nella condizione, anche se omonimi.

Esempio. $S1 \bowtie_{S1.sid < R1.sid} R1$: delle 6 combinazioni possibili, solo 2 soddisfano la condizione $sid_{S1} < sid_{R1}$: $\langle 22, dustin, 7, 45.0, 58, 103, 11/12/96 \rangle$ e $\langle 31, lubber, 8, 55.5, 58, 103, 11/12/96 \rangle$.

3.8.2 Join Naturale

Espressione: $R_1 \bowtie R_2$	
Schema	$R_1(X_1), R_2(X_2) \rightarrow X_1 \cup X_2$
Istanza	$r_1, r_2 \rightarrow r_1 \bowtie r_2 = \{t \mid t[X_1] \in r_1 \text{ AND } t[X_2] \in r_2\}$

Tabella 5: Definizione formale del join naturale

Ogni tupla del risultato combina (*match*) una tupla di r_1 con una di r_2 sulla base dell'**uguaglianza** dei valori degli attributi comuni ($X_1 \cap X_2$); questi attributi compaiono **una sola volta** nello schema risultato.

Osservazioni:

- una tupla che non trova corrispondenza nell'altra relazione si dice **dangling**;
- la cardinalità del join è compresa tra 0 e $|r_1| \cdot |r_2|$;
- se il join è eseguito su una superchiave di $R_1(X_1)$, allora $|r_1 \bowtie r_2| \leq |r_2|$;
- se $X_1 \cap X_2$ è chiave primaria di $R_1(X_1)$ e chiave esterna in $R_2(X_2)$ (vincolo di integrità referenziale), allora $|r_1 \bowtie r_2| = |r_2|$.

3.8.3 Equijoin

È la forma di join più comunemente utilizzata: la condizione c include **soltanto** confronti di uguaglianza (=).

Esempio. $S1 \bowtie_{sid=sid} R1$: risultato $\{\langle 22, \text{dustin}, 7, 45.0, 101, 10/10/96 \rangle, \langle 58, \text{rusty}, 10, 35.0, 103, 11/12/96 \rangle\}$ (la tupla con $sid = 31$ è *dangling*: nessuna corrispondenza in R1).

3.9 Divisione ($\div$)

Dati $A(x, y)$ e $B(y)$, $A \div B$ restituisce l'insieme delle tuple x tali che, per **ogni** tupla y in B , la coppia (x, y) compare in A . Caso d'uso tipico: "trovare i sailor che hanno prenotato **tutte** le barche in un certo insieme".

Esempio. Dato $A(sno, pno)$ con le coppie $\{s_1p_1, s_1p_2, s_1p_3, s_1p_4, s_2p_1, s_2p_2, s_3p_2, s_4p_2, s_4p_4\}$:

- $B_1 = \{p_2\} \Rightarrow A/B_1 = \{s_1, s_2, s_3, s_4\}$ (tutti prenotano p_2);
- $B_2 = \{p_2, p_4\} \Rightarrow A/B_2 = \{s_1, s_4\}$ (solo loro prenotano **sia** p_2 **sia** p_4);
- $B_3 = \{p_1, p_2, p_4\} \Rightarrow A/B_3 = \{s_1\}$ (solo s_1 prenota tutte e tre).

3.10 Outer Join

A differenza del join (naturale o theta), l'**outer join** conserva anche le tuple *dangling*, riempiendo con NULL gli attributi mancanti:

- **LEFT OUTER JOIN**: conserva le tuple *dangling* della relazione di sinistra;
- **RIGHT OUTER JOIN**: conserva le tuple *dangling* della relazione di destra;
- **FULL OUTER JOIN**: conserva le tuple *dangling* di entrambe le relazioni.

3.11 Espressioni ed Alberi di Query

Un'espressione di algebra relazionale può essere rappresentata come un **albero di query**: le foglie sono le relazioni di base, i nodi interni sono gli operatori, e la valutazione procede dal basso verso l'alto. Espressioni algebricamente equivalenti (stesso risultato per ogni istanza valida) possono corrispondere ad alberi diversi con costi di esecuzione molto differenti: è la base per l'**ottimizzazione delle query**, approfondita quando si tratterà l'analisi dei costi.